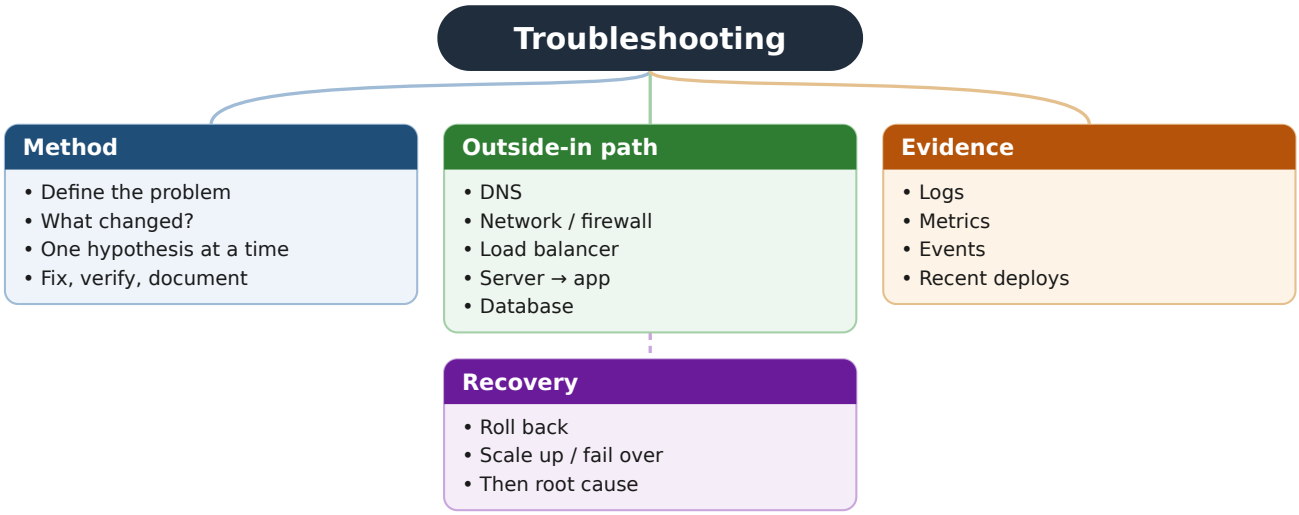
17. Troubleshooting

DevOps, Cloud & Platform Engineering · Complete Notes · Topic 17 of 18 · Thirupathi Keloth · v2.0

Contents

17.0 Why a separate troubleshooting book
17.1 The method
17.2 The toolkit by layer
17.3 Linux host playbooks
17.4 Network playbooks
17.5 Web applications and HTTP
17.6 Git problems
17.7 Docker problems
17.8 Kubernetes quick decision tree
17.9 AWS playbooks
17.10 Terraform problems
17.11 Ansible problems
17.12 CI/CD problems
17.13 Scenario drills
17.14 Interview questions
17.15 Self-check



17.0 Why a separate troubleshooting book

Each topic's notes end with their own troubleshooting table. Real incidents don't respect topic boundaries: "the website is slow" might be DNS, a full disk, a bad deploy, a database lock, an exhausted NAT gateway port range, or CPU throttling in Kubernetes. This book gives you **one method** and **cross-domain playbooks**, the way senior engineers actually work through problems under pressure.

17.1 The method

```
1. DEFINE      What exactly is wrong? For whom? Since when? How bad? (scope + impact + start time)
2. STABILIZE   If users are hurting: mitigate first (roll back, fail over, scale, disable feature)
3. WHAT CHANGED Deploys, config, infra, traffic, certificates, dependencies, time (cron, DST)
4. HYPOTHESIZE Based on evidence, list likely causes, most probable and cheapest to check first
5. TEST        One change or one check at a time; record what you did and saw (timestamps)
6. NARROW      Divide and conquer: which layer, which component, which hop
7. FIX + VERIFY Fix the cause, confirm with the same signal that showed the problem
8. PREVENT     Postmortem actions: alert, test, guardrail, runbook, automation
```

Questions that solve half of all incidents

Question	Why
What changed?	Most outages follow a change. Check deploy history, Git log of the config repo, Terraform applies, CloudTrail
Is it everyone or some users?	Some = one region/AZ/node/pod/version/browser/customer: compare good vs bad
Is it all requests or some endpoints?	One endpoint = code/dependency; all = infrastructure/network/capacity
Did it start suddenly or gradually?	Sudden = change or failure; gradual = leak, growth, saturation, certificate/disk countdown
Can I reproduce it?	Reproduction turns guessing into testing
What do the metrics, logs, traces and events say at the start time?	Evidence over intuition

Divide by layers (bottom up or top down)

```
User/browser ⟶ DNS ⟶ CDN/WAF ⟶ Load balancer ⟶ Ingress/Proxy ⟶ Service ⟶ Pod/Process
     ⟶ Application code ⟶ Dependencies (DB, cache, queue, external APIs)
Underneath everything: compute (CPU, memory, disk, network), OS, cloud quotas/limits, identity (IAM)
```

Test at a hop to cut the problem in half: if `curl` from inside the cluster to the Service works but from the internet it doesn't, everything from the Service inward is fine.

Anti-patterns under pressure

- Changing several things at once (you'll never know what fixed or broke it).
- Restarting before capturing evidence (logs, heap dumps, `ps`, `ss` output). Capture first, then restart if needed.
- Trusting assumptions ("the config is definitely right"): verify.
- Debugging alone for hours during a SEV1: escalate, declare an incident, get an incident commander.
- Fixing in production by hand without putting the fix back into code (GitOps will revert it, or the next deploy will).

17.2 The toolkit by layer

Layer	Commands / tools
Process / CPU / memory	`top` / `htop`, `ps aux --sort=-%cpu`, `free -h`, `vmstat 1`, `pidstat 1`, `/proc/<pid>/status`
Disk	`df -h`, `df -i`, `du -xh --max-depth=1 /`, `iostat -xz 1`, `lsof +L1`
Logs	`journalctl -u nginx --since "10 min ago"`, `journalctl -p err -b`, `dmesg -T`, `/var/log/*`
Services	`systemctl status/restart/cat nginx`, `systemctl --failed`
Network config	`ip a`, `ip r`, `ip neigh`, `resolvectl status`, `cat /etc/resolv.conf`
Connectivity	`ping`, `traceroute` / `mtr`, `nc -zv host port`, `curl -v`, `telnet`
Sockets	`ss -tulpn` (listening), `ss -tanp state established`, `ss -s`
DNS	`dig name`, `dig +trace name`, `dig @8.8.8.8 name`, `nslookup`

Layer	Commands / tools
TLS	`openssl s_client -connect host:443 -servername host`, `curl -vI https://...`
Packets	`tcpdump -i any port 443 -nn`, Wireshark
HTTP	`curl -w '%{http_code} %{time_connect} %{time_starttransfer} %{time_total}\n' -o /dev/null -s URL`
Containers	`docker ps -a`, `docker logs --tail 100 -f`, `docker inspect`, `docker exec`, `docker stats`
Kubernetes	`kubectl get/describe/logs/events/top/debug` (Kubernetes notes 12.18)
AWS	Console health/status checks, CloudWatch metrics/logs, CloudTrail, VPC Reachability Analyzer, `aws sts get-caller-identity`, Systems Manager Session Manager
Observability	Grafana dashboards, PromQL, LogQL/Logs Insights, traces

Linux performance in 60 seconds (Brendan Gregg's checklist)

```
uptime                    # load averages: trend over 1, 5, 15 minutes
dmesg -T | tail           # kernel errors: OOM killer, disk/network errors
vmstat 1 5                # r (run queue), free memory, si/so (swapping), us/sy/wa/st CPU
mpstat -P ALL 1 3         # per-CPU balance: one hot CPU = single-threaded bottleneck
pidstat 1 3               # which processes use CPU
iostat -xz 1 3            # disk: %util, await (latency), r/s w/s
free -m                   # memory and page cache
sar -n DEV 1 3            # network throughput per interface
sar -n TCP,ETCP 1 3       # TCP connections, retransmits
top                       # overview
```

17.3 Linux host playbooks

High CPU

1. `top` (press `1` for per-CPU, `P` to sort): which process? `us` (user code), `sy` (kernel), `wa` (waiting for disk), `st` (stolen by hypervisor: noisy neighbor or burstable credits exhausted).
2. Process identified → is it expected (traffic growth) or abnormal (infinite loop, runaway job, crypto miner)?
3. Look inside: `pidstat -t -p <pid> 1` (threads), application profiler, `perf top`, Java `jstack`, Python `py-spy top --pid <pid>`.
4. On AWS t-family instances: CPU credit balance at zero → throttled to baseline: switch to unlimited mode or a non-burstable type.

High load average but CPU idle

Load counts processes **running and waiting** (including uninterruptible disk I/O, state `D`). High load + low CPU = processes blocked on I/O: `vmstat` (`wa`, `b` column), `iostat -xz 1` (high `await`, `%util` near 100%), `ps -eo state,pid,cmd | grep '^D'`. Causes: slow/saturated disk (EBS throughput/IOPS limits, gp2 burst credits), NFS/EFS hangs.

Memory pressure and OOM

```
free -h                                   # "available" is what matters, not "free"
ps aux --sort=-%mem | head
dmesg -T | grep -i -E "killed process|out of memory"
journalctl -k | grep -i oom
cat /proc/meminfo | grep -E "MemAvailable|Slab|Shmem"
```

Linux uses free RAM for page cache: low "free" is normal. Real trouble: low **available**, swapping (`si/so` in `vmstat`), the OOM killer in `dmesg`. Fix the leak (heap dumps, profiler), set limits (systemd `MemoryMax=`, container limits), right-size the instance, tune runtime heaps (JVM `-Xmx` must leave room for non-heap memory).

Disk full

```
df -h                                    # which filesystem?
df -i                                    # inodes exhausted? (millions of tiny files)
sudo du -xh --max-depth=1 / 2>/dev/null | sort -h | tail -15   # biggest directories
sudo lsof +L1                            # deleted files still held open by a process
sudo journalctl --vacuum-size=500M       # shrink the systemd journal
docker system df && docker system prune  # container images/volumes (careful in prod)
```

Classic trap: you deleted a huge log but `df` still shows full, because the process still holds the file open. Restart/reload that process, or truncate instead of deleting: `: > /var/log/app/big.log`. Prevent with **logrotate**, journal limits, monitoring with `predict_linear` alerts, and growing EBS volumes online (`growpart` + `resize2fs` / `xfs_growfs`).

A service won't start

```
systemctl status myapp                   # state, last log lines, exit code
journalctl -u myapp -n 100 --no-pager    # full recent logs
systemctl cat myapp                      # the unit file actually in use
sudo -u myapp /opt/myapp/bin/start       # run as the service user to see the real error
namei -l /opt/myapp/config.yml           # permissions along the whole path
ss -tulpn | grep :8080                   # port already in use?
```

Common causes: wrong path/permissions, missing environment file, port in use, config syntax error (`nginx -t`, `sshd -t`), dependency not started, SELinux/AppArmor denial (`ausearch -m avc -ts recent`, `getenforce`), `ExecStart` not absolute.

"Too many open files"

`ulimit -n` (shell), `cat /proc/<pid>/limits` (actual process), `ls /proc/<pid>/fd | wc -l`. Raise with systemd `LimitNOFILE=65535` in the unit (not `/etc/security/limits.conf` for services). Also check for connection leaks in the app.

Other host classics

Symptom	Check
Clock wrong / TLS and auth failures ("certificate not yet valid", Kerberos/AWS signature errors)	`timedatectl`, chrony (`chronyc tracking`); AWS Time Sync at 169.254.169.123
Can't SSH	Security group/NACL, sshd running, `~/.ssh` permissions (700, key 600), wrong user (`ec2-user` vs `ubuntu`), disk full, instance status checks; use **SSM Session Manager** instead
`Permission denied` despite correct mode	Ownership, parent directory `x` bit, ACLs (`getfacl`), SELinux context (`ls -Z`, `restorecon`), mount options (`noexec`)
Zombie processes	Parent isn't reaping children: fix/restart the parent; zombies themselves use no CPU/memory
Cron job didn't run	`grep CRON /var/log/syslog` or `journalctl -u cron`, PATH differences in cron, missing newline at the end of the crontab, timezone

17.4 Network playbooks

"I can't connect": walk the layers

```
# 1. Name resolution
dig +short api.example.com                    # nothing? -> DNS problem
# 2. Route to the network
ip route get 10.0.20.15                        # which interface / gateway?
traceroute -n 10.0.20.15                       # where does it stop?
# 3. Port reachable?
nc -zv -w 3 10.0.20.15 5432                    # succeeded / refused / timed out
# 4. Service listening on the server (run there)
ss -tulpn | grep 5432                          # listening on 0.0.0.0 or only 127.0.0.1?
# 5. Application protocol
curl -sv https://api.example.com/healthz       # TLS handshake, HTTP status, headers
```

Read the error: refused vs timeout vs reset

Error	Meaning	Typical causes
Connection refused	Packet reached the host; nothing listening on that port (TCP RST)	Service down, wrong port, listening on `127.0.0.1` only, container port not published
Connection timed out	No answer at all: packets dropped	Security group/NACL/firewall drop, wrong route, host down, wrong IP, no NAT/IGW for egress
Connection reset by peer	Connection was established, then killed	App crashed mid-request, proxy idle timeout, TLS mismatch, load balancer deregistration
No route to host	Routing or ARP failure (or ICMP reject by firewall)	Wrong subnet/route table, host down on the LAN
`Name or service not known` / NXDOMAIN	DNS has no answer	Typo, missing record, wrong DNS server, private zone not associated with the VPC
`SERVFAIL`	DNS server failed to resolve	DNSSEC problems, upstream DNS broken

AWS network checklist (instance A can't reach B)

Check	Where
Security group on **B** allows inbound from A's SG/CIDR on the port	EC2 → Security groups (stateful: return traffic automatic)
Security group on **A** allows outbound	Usually all outbound allowed by default
NACLs on both subnets allow the port **and** ephemeral return ports (1024–65535)	VPC → Network ACLs (stateless)
Route tables: local route within VPC; peering/TGW routes both directions	VPC → Route tables
Internet egress: public subnet needs IGW route + public IP; private subnet needs NAT gateway route	Route tables, NAT status
Host firewall (iptables/firewalld/ufw)	On the instance
DNS: private hosted zone associated with VPC; `enableDnsSupport/Hostnames` on	Route 53, VPC settings
VPC Reachability Analyzer: tells you exactly which component blocks the path	VPC → Reachability Analyzer

Check	Where
VPC Flow Logs: `REJECT` entries show drops	CloudWatch Logs / S3

DNS problems

```
dig api.example.com                          # answer, TTL, which server answered
dig @1.1.1.1 api.example.com                 # compare with a public resolver
dig +trace api.example.com                   # walk delegation from the root
dig NS example.com ; dig SOA example.com     # delegation and zone serial
getent hosts api.example.com                 # what the OS resolver (incl. /etc/hosts) returns
```

Traps: stale caches after a change (wait for the old **TTL**; lower TTL *before* migrations), `/etc/hosts` overrides, split-horizon DNS (different answers inside vs outside the VPC), CNAME at the zone apex (use Route 53 alias records), Kubernetes `ndots` lookups (12.7).

TLS problems

```
openssl s_client -connect shop.example.com:443 -servername shop.example.com </dev/null \
  | openssl x509 -noout -subject -issuer -dates -ext subjectAltName
curl -vI https://shop.example.com 2>&1 | grep -E "SSL|subject|expire|issuer"
```

Error	Cause
`certificate has expired`	Renewal failed (cert-manager, ACM DNS validation record removed)
`hostname mismatch` / `SAN`	Certificate doesn't cover this name (`www` vs apex, new subdomain)
`unable to get local issuer certificate`	Server doesn't send the intermediate chain, or client lacks the CA (corporate proxy CAs)
`handshake failure` / `protocol version`	No common TLS version/cipher; SNI missing
Wrong certificate served	Missing SNI (`-servername`), default certificate on the load balancer listener

Slow network

- Measure where time goes: `curl -w` (DNS, connect, TLS, first byte), `mtr` (per-hop loss and latency).
- Packet loss/retransmits: `sar -n ETCP 1`, `ss -ti` (retrans), `netstat -s | grep retrans`.
- **MTU problems** (VPN, Transit Gateway, jumbo frames): small requests work, large ones hang. Test with `ping -M do -s 1472 host` and lower until it passes.
- Cross-AZ or cross-region traffic adds latency and cost; NAT gateway throughput and **port exhaustion** (`ErrorPortAllocation` metric) for many outbound connections to one destination.

17.5 Web applications and HTTP

Status codes as clues

Code	Meaning	Where to look
400 Bad Request	Malformed request	Client payload, headers too large, API validation
401 Unauthorized	Not authenticated	Missing/expired token, clock skew, wrong auth header

Code	Meaning	Where to look
403 Forbidden	Authenticated but not allowed	Authorization rules, WAF block, S3/IAM policy, CORS-related blocks in browsers
404 Not Found	Path doesn't exist here	Wrong route/ingress path rule, missing file, wrong host header/virtual host
405 / 413 / 429	Method not allowed / body too large / rate limited	Proxy `client_max_body_size`, API Gateway/WAF throttling
500 Internal Server Error	Application threw an unhandled error	**Application logs** and traces
502 Bad Gateway	Proxy/LB got an invalid response or connection error from the backend	Backend crashed or closed connection, wrong target port, keep-alive timeout mismatch (backend idle timeout shorter than LB's)
503 Service Unavailable	No healthy backend or service overloaded	Target group has no healthy targets, Kubernetes Service with no endpoints, maintenance mode, circuit breaker open
504 Gateway Timeout	Backend didn't answer in time	Slow queries, dependency timeouts, LB/ingress timeout too short for the operation, backend unreachable (security group)

502/503/504 behind an ALB: decision path

```
Check ALB metrics: HTTPCode_ELB_5XX (generated by the ALB) vs HTTPCode_Target_5XX (from your app)
├── Target 5xx → application errors → app logs/traces
└── ELB 5xx
    ├── 503 → target group: healthy targets = 0? → health check path/port/codes, app startup,
    │                                               security group from ALB to targets
    ├── 502 → targets resetting connections → app crashes, keep-alive timeout < ALB idle timeout
    │                                          (60s default): set app keep-alive > 60s
    └── 504 → target too slow / unreachable → TargetResponseTime, DB latency, ALB idle timeout
```

Latency spikes

1. Is it all endpoints or one? One → code path or its dependency (trace it). All → shared resource.
2. Saturation: CPU (and **CPU throttling** in containers), memory/GC pauses, thread or worker pool exhaustion, **database connection pool** exhaustion (requests wait for a connection).
3. Dependencies: database slow queries (`pg_stat_statements`, slow query log, locks: `pg_locks`), cache hit ratio drop (cold cache after deploy/flush), external API latency.
4. Infrastructure: noisy neighbor/steal time, EBS throttling, network retransmits, DNS resolution delays, cross-AZ hops.
5. Traffic shape: a retry storm (clients retrying without backoff multiply the load), one big customer, a bot.

Memory leak pattern

Memory grows steadily after each deploy until OOMKilled/restart (sawtooth graph). Confirm with the memory metric over days, take heap dumps at two points and compare (Java: `jmap`/JFR; Python: `tracemalloc`, `memray`; Node: heap snapshots). Mitigate with restarts/limits while fixing; don't "fix" with bigger instances only.

Database connection problems

Symptom	Cause
`too many connections`	Each pod's pool × number of pods > DB `max_connections`. Use smaller pools, a pooler (PgBouncer, RDS Proxy)
Timeouts connecting	Security group, wrong endpoint (writer vs reader), DB failover in progress, DNS caching of old IP
Auth failed after rotation	App cached old password: reload secrets on rotation, use RDS Proxy/IAM auth
Slow after a deploy	New query without an index, ORM N+1 queries: check `EXPLAIN ANALYZE`

17.6 Git problems

Problem	Solution
Committed to the wrong branch (not pushed)	`git switch -c right-branch` (keeps the commit), then on the old branch `git reset --hard HEAD~1`
Undo the last commit, keep changes	`git reset --soft HEAD~1`
Undo a pushed commit safely	`git revert <sha>` (new commit; history unchanged)
"Lost" commits after a reset/rebase	`git reflog` → `git switch -c rescue <sha>`
Merge conflict	`git status` → edit the conflict markers → `git add` → `git commit` (or `git rebase --continue`); `git merge --abort` to back out
`rejected (non-fast-forward)` on push	Remote has new commits: `git pull --rebase` then push; never force-push shared branches (`--force-with-lease` only on your own)
Detached HEAD	You checked out a commit/tag: `git switch -c new-branch` to keep work, or `git switch main`
Secret committed	Rotate the secret **first**; then remove from history (`git filter-repo`) if needed (DevSecOps 14.3)
Huge file rejected by GitHub	Remove from history, use Git LFS or artifact storage
Line ending noise (Windows)	`git config --global core.autocrlf true` on Windows; `.gitattributes` with `* text=auto`
Authentication failed	HTTPS needs a token (not password); SSH: `ssh -T git@github.com`, correct key loaded (`ssh-add -l`)

17.7 Docker problems

Symptom	Check / fix
Container exits immediately	`docker logs <c>`, `docker inspect <c> --format '{{.State.ExitCode}}'`; the main process must run in the foreground
Exit code 137	OOM-killed (`docker inspect` → `OOMKilled: true`) or `docker kill`; raise memory limit or fix the leak
Can't reach the app from the host	Port not published (`-p 8080:80`), app listening on `127.0.0.1` inside the container (must be `0.0.0.0`), wrong container port

Symptom	Check / fix
Containers can't reach each other by name	Use a user-defined network (`docker network create`); the default bridge has no DNS
`permission denied` on a bind mount	UID/GID mismatch between host files and the container user; SELinux `:z` label
Build is slow / always rebuilds everything	Layer order: copy dependency files and install before copying source; `.dockerignore`
`no space left on device`	`docker system df` , prune unused images, build cache (`docker builder prune`)
Image pull rate limited (429)	Authenticate to Docker Hub, use a mirror/ECR pull-through cache
`exec format error`	Image built for another CPU architecture (arm64 vs amd64): `docker buildx build --platform`

17.8 Kubernetes quick decision tree

Full tables in the Kubernetes notes (12.18). The fastest path:

```
kubectl get pods -n <ns>
├── Pending ──────────────► describe pod → Events: resources / affinity / taints / PVC
├── ContainerCreating ────► describe pod → mount, secret/configmap missing, CNI IPs
├── ImagePullBackOff ─────► image name/tag, registry auth (ECR permissions), network to registry
├── CrashLoopBackOff ─────► logs --previous; exit code: 137 = OOM, 1 = app error, 127 = no command
├── Running 0/1 READY ────► readiness probe failing → probe path/port, app dependencies
└── Running 1/1 but broken ► Service selector ↔ labels, EndpointSlices, targetPort,
                             NetworkPolicy, Ingress rules/class, DNS name
Nodes: kubectl get nodes → NotReady → describe node (pressure conditions), kubelet logs
Recent change? kubectl rollout history; Argo CD history;
               kubectl get events -A --sort-by=.lastTimestamp
```

17.9 AWS playbooks

IAM `AccessDenied`

Policy evaluation logic (in order):

```
1. Explicit DENY anywhere (identity policy, resource policy, SCP, RCP, boundary, session) ─► DENIED
2. Organizations SCPs / RCPs allow it?                          no ─► DENIED
3. Resource-based policy allows it (same account)?             may grant access by itself
4. Identity-based policy allows it?                            no (and no resource grant) ─► DENIED
5. Permissions boundary allows it? Session policy allows it?   no ─► DENIED
Default: implicit DENY. Cross-account: BOTH the identity policy AND the resource policy must allow.
```

```
aws sts get-caller-identity                    # who am I really? (wrong profile is very common)
aws sts decode-authorization-message --encoded-message "<message from the error>" \
  --query DecodedMessage --output text | jq .   # which statement denied it (needs permission)
aws iam simulate-principal-policy --policy-source-arn arn:aws:iam::123456789012:role/app \
  --action-names s3:GetObject --resource-arns arn:aws:s3:::acme-shop-uploads-prod/a.txt
```

Also check CloudTrail for the failed event (`errorCode: AccessDenied`): it shows the exact principal, action and resource. Frequent causes: wrong resource ARN (bucket ARN vs `bucket/*` object ARN), missing KMS key permission for encrypted objects, SCP region restriction, condition keys (source VPC endpoint, MFA, tags), and the instance/pod using a different role than you think.

S3 403 checklist

Identity policy → bucket policy (explicit denies, `aws:SecureTransport`, VPC endpoint conditions) → Block Public Access (for public access) → **Object Ownership**/ACLs (objects uploaded by another account) → KMS key policy → the object actually exists (missing `s3:ListBucket` turns 404 into 403).

EC2 instance unreachable

Check	Tool
Instance state, **system** status check (AWS hardware/network) vs **instance** status check (your OS: kernel, disk full, network config)	EC2 console / `aws ec2 describe-instance-status`
Console output / screenshot	EC2 → Monitor and troubleshoot → Get system log / instance screenshot
Security group, NACL, route, public IP/EIP	Reachability Analyzer
SSH key/user, sshd	SSM Session Manager (requires SSM agent + instance profile + network path to SSM endpoints)
Recover a broken OS	EC2 Serial Console, or stop → detach root volume → attach to a rescue instance → fix → reattach

Other AWS classics

Symptom	Likely cause
Private instance can't reach the internet / AWS APIs	No NAT gateway route, NAT in a subnet without IGW route, or missing VPC endpoints; NACL blocks ephemeral ports
Lambda timeout	Function in a VPC without NAT/endpoints for the service it calls; downstream slow; timeout too low; cold starts
Lambda `AccessDenied` / KMS errors	Execution role missing permission; environment variable encryption key policy
RDS connection refused/timeout	Security group from app SG, wrong endpoint, public access off (correct for prod), DB stopped/storage full
ALB health checks failing	Wrong path/port/success codes, security group from ALB to targets, app startup slower than the grace period
Auto Scaling keeps replacing instances	Health check failing during boot: increase grace period, fix user data errors (`/var/log/cloud-init-output.log`)
CloudFront 403/502	Origin access control/bucket policy, origin protocol/certificate mismatch, origin timeout
Sudden cost spike	Cost Explorer by service/usage type → NAT gateway data, data transfer, forgotten instances/volumes/load balancers, logging volume, leaked keys used for mining. Set **budgets with alerts**
`LimitExceeded` / `InsufficientInstanceCapacity`	Service quotas (request increase); capacity: try other instance types/AZs

17.10 Terraform problems

Problem	Solution
`Error acquiring the state lock`	Another run is active, or a crashed run left the lock: confirm nobody is running, then `terraform force-unlock <LOCK_ID>`
Plan wants to recreate a resource unexpectedly	Read the `# forces replacement` line: an immutable argument changed; use `lifecycle { prevent_destroy = true }` for critical resources, `moved` blocks for renames
Drift (changes made in the console)	`terraform plan -refresh-only` to see it; fix in code or accept with `apply -refresh-only`
Resource exists but not in state	`import` block (or `terraform import`) then plan until no changes
`No valid credential sources found`	AWS profile/env vars/SSO login (`aws sso login`), `aws sts get-caller-identity`
Provider/version errors after an upgrade	Constrain versions, commit `.terraform.lock.hcl`, `terraform init -upgrade` deliberately
`Cycle` error	Two resources reference each other: split rules (e.g. separate `aws_security_group_rule`), remove the circular dependency
Destroy fails (dependency violation)	Resources created outside Terraform depend on it (ENIs from Lambda/EKS, objects in a bucket): find and remove them
State corruption/loss	Restore a previous state version (S3 versioning), never edit state by hand: use `terraform state mv/rm`
`count` / `for_each` "value unknown until apply"	Keys must be known at plan time: use static keys, not resource attributes

17.11 Ansible problems

Problem	Solution
`UNREACHABLE`	SSH: host, user, key, port, security group; test with `ssh -i key user@host`; `ansible all -m ping -vvv`
`Missing sudo password` / permission denied	`become: true`, `--ask-become-pass`, or passwordless sudo for the automation user
Module fails on Python	Wrong `ansible_python_interpreter`; missing Python libraries on the target (e.g. `python3-apt`)
Task always "changed"	Using `command` / `shell` without `creates` / `changed_when`; prefer real modules
Variable has an unexpected value	Variable precedence: `ansible -m debug -a "var=myvar"`, `ansible-inventory --host h1`
Handler didn't run	Task wasn't "changed", or play failed before handlers: `--force-handlers`, `meta: flush_handlers`
Slow runs	`gather_facts: false` when not needed, fact caching, `pipelining = True`, more `forks`

17.12 CI/CD problems

Problem	Check
Works locally, fails in CI	Different versions (pin tool versions), missing env vars/secrets, OS/architecture differences, cached state locally, test order dependence
Flaky tests	Time/order/network dependence; rerun once to confirm, then fix or quarantine *with a ticket* (never silently skip)
Secrets empty	Fork PRs don't receive secrets; environment-scoped secret used in a job without `environment:`; wrong name
OIDC `Not authorized to perform sts:AssumeRoleWithWebIdentity`	Trust policy `sub` condition doesn't match repo/branch/environment; missing `id-token: write`
Docker push denied	Registry login step, repository doesn't exist (ECR needs it created), IAM permissions
Pipeline slow	Cache dependencies and Docker layers, parallelize jobs, run only affected tests/paths
Deploy "succeeded" but old version runs	Same mutable tag (`latest`) with `IfNotPresent` , GitOps repo not updated, Argo CD not synced, wrong cluster/namespace

17.13 Scenario drills

Practice talking through these out loud: hypothesis → check → finding → fix → prevention.

#	Scenario	Key path to the answer
1	Website down right after a deploy	Roll back first; compare versions; check readiness and errors of new pods
2	Site slow every day at 02:00	Scheduled job: backups, cron, batch jobs, log rotation; correlate timing
3	Disk 100% on a server, `du` shows little	Deleted-but-open files (`lsof +L1`), or another mount hidden under a mount point
4	Some users get errors, others don't	Compare: region/AZ, one bad node/pod, version skew, specific customer data, DNS cache
5	Pods `CrashLoopBackOff` after a config change	`logs --previous` , missing env/key; revert config in Git
6	`connection timed out` from app to RDS	Security group inbound from the app's SG, subnets/route, correct endpoint
7	502s from the ALB a few times per minute	Keep-alive timeout mismatch, pods terminating without draining (preStop/SIGTERM)
8	AWS bill doubled this month	Cost Explorer by usage type; NAT data, idle resources, logs; budgets + tags
9	Terraform plan wants to destroy the production database	Stop; read why (forces replacement / renamed resource); `moved` block or fix argument; `prevent_destroy`
10	Certificate expired on Saturday	Renew/re-issue now; then automation + expiry alerts 14+ days ahead

#	Scenario	Key path to the answer
11	Memory grows until restart every 3 days	Leak: heap dumps, recent dependency changes; temporary restart schedule while fixing
12	Cannot SSH to an EC2 instance after a reboot	Status checks, console output: fstab error, disk full; Session Manager or rescue volume
13	Kubernetes node `NotReady`	Kubelet logs, disk/memory pressure, CNI, instance health; cordon/drain/replace
14	CI suddenly fails on all branches	External change: base image/tool version, expired token, registry outage, runner image update
15	DNS change not visible to users after an hour	Old TTL, resolver caches, wrong zone/delegation, private vs public zone
16	Latency up 3× but CPU low	Waiting: DB locks/slow queries, connection pool exhaustion, external API, CPU throttling, network
17	`AccessDenied` for a role that "has admin"	SCP, permission boundary, explicit deny in bucket/KMS policy, different role actually used
18	Queue backlog growing	Consumers down/slow, poison messages (DLQ), scaling not triggered, downstream throttling
19	Deploy succeeded, users still see the old version	CDN/browser cache, old pods still serving, wrong environment, mutable tag
20	Monitoring shows everything green, users complain	Monitoring the wrong thing (no user-facing SLI), synthetic checks missing, alerting broken (dead man's switch)

17.14 Interview questions

1. Walk me through how you troubleshoot "the website is slow".
2. What do you do first in an outage: find the root cause or mitigate? Why?
3. High load average but low CPU usage: what's happening?
4. A disk is full, you deleted logs, still full: why?
5. Connection refused vs connection timed out: what does each tell you?
6. How do you troubleshoot DNS resolution problems?
7. 502 vs 503 vs 504 from a load balancer: causes of each.
8. An EC2 instance in a private subnet can't reach the internet: checks in order.
9. Explain IAM policy evaluation. How do you debug `AccessDenied`?
10. A pod is in `CrashLoopBackOff`: steps.
11. Terraform wants to replace a production database: what do you do?
12. How do you troubleshoot an intermittent problem you can't reproduce?
13. Tell me about the hardest production issue you solved (use: situation, impact, method, root cause, fix, prevention).
14. How do you avoid making an incident worse while troubleshooting?
15. What do you put in a runbook?

17.15 Self-check

- [ ] I follow the method: define, stabilize, what changed, hypothesize, test, narrow, fix, prevent.
- [ ] I can run the 60-second Linux performance checklist and interpret it.
- [ ] I can solve the host playbooks: CPU, load, memory/OOM, disk full, services, file limits.
- [ ] I can walk a connection problem layer by layer and interpret refused/timeout/reset.

- [ ] I can debug DNS and TLS with dig and openssl.
- [ ] I can map HTTP 5xx errors behind a load balancer to causes.
- [ ] I can recover from common Git, Docker, Terraform, Ansible and CI/CD problems.
- [ ] I can debug IAM AccessDenied, S3 403 and EC2/VPC connectivity on AWS.
- [ ] I can talk through all 20 scenario drills confidently.

REMEMBER

- The first question is always **"What changed?"**
- Change one thing at a time and write down what you did.